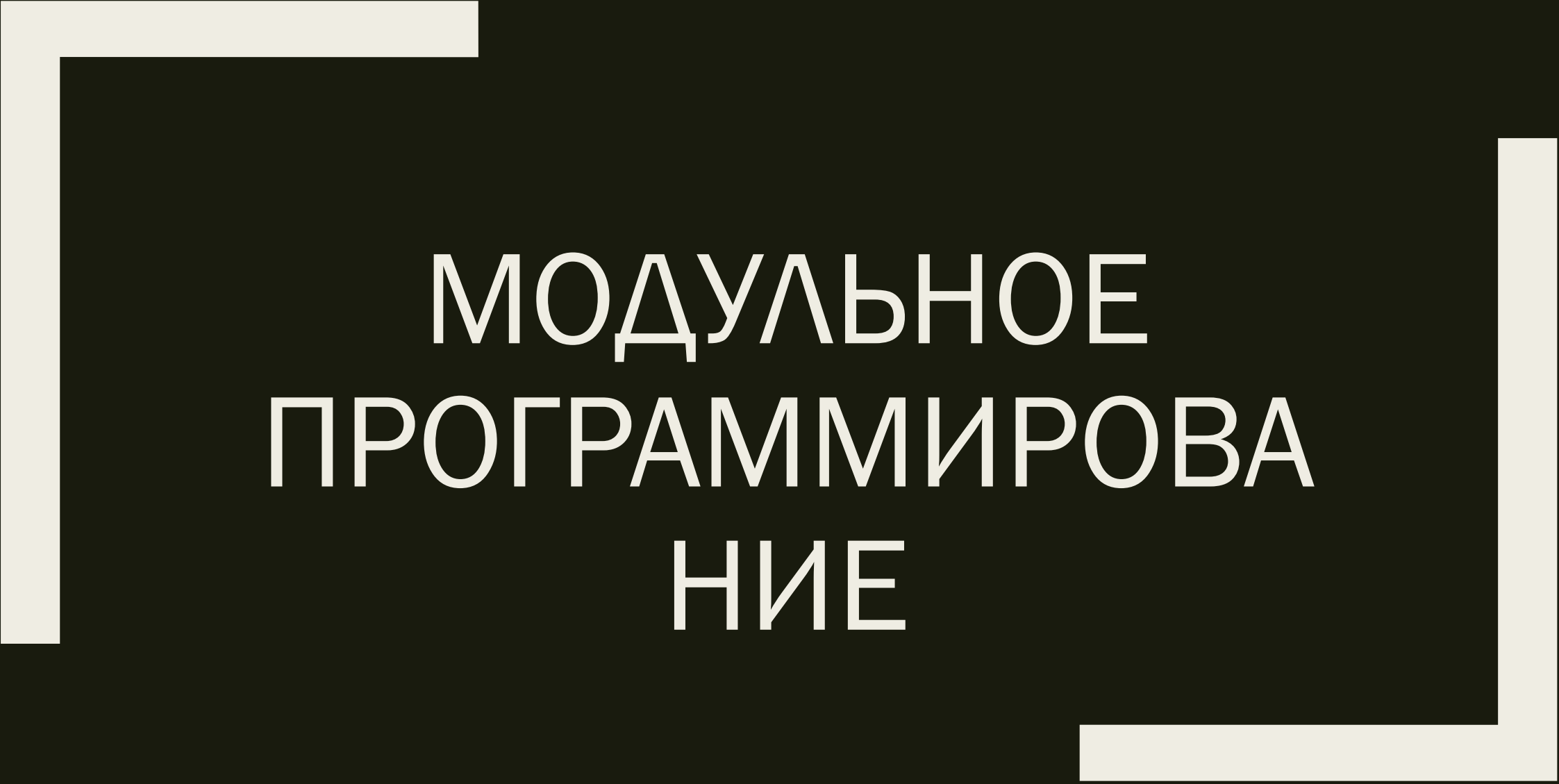


A decorative frame composed of two thick, white L-shaped lines. One line starts at the top-left and extends horizontally to the right, then vertically downwards. The other line starts at the bottom-right and extends horizontally to the left, then vertically upwards. They meet at the center, framing the text.

МОДУЛЬНОЕ ПРОГРАММИРОВА НИЕ

Основные понятия

Модульное программирование – это способ программирования, при котором вся программа разбивается на группу компонентов, называемых модулями, причем каждый из них имеет свой контролируемый размер, четкое назначение и детально проработанный интерфейс с внешней средой.

Альтернатива модульности – монолитная программа.

Основные концепции

В основе модульного программирования лежат три основные концепции:

– Принцип утаивания информации

- При разработке программ формируется список проектных решений, которые трудно понять или которые могут меняться.
- Затем определяются отдельные модули, каждый из которых реализует одно из указанных решений.
- Программы должны использовать модули без каких-либо предварительных знаний об их внутренней структуре. Примерами удачных модулей могут служить стандартные процедуры.

– Сборочное программирование

Модули – это программные «кирпичи», из которых строится программа.

Основные концепции

– Аксиома модульности

Модуль – независимая программная единица, служащая для выполнения некоторой определенной функции программы и для связи с остальной частью программы.

Программная единица должна удовлетворять следующим условиям:

- Блочность организации
- Синтаксическая обособленность
- Семантическая независимость
- Общность данных
- Полнота определения

Характеристики модуля

- Размер модуля
- Связность (прочность) модуля
- Сцепление модуля с другими модулями
- Рутинность (независимость от предыдущих обращений) модуля

СВЯЗНОСТЬ МОДУЛЯ

Связность модуля – мера независимости его частей. Чем выше связность, тем больше связей он «упрятывает» в себе

- Типы связности:

- *Функциональная*
- *Последовательная*
- *Информационная*

Типы связности модуля

- **Функциональная**

Модуль с функциональной связностью реализует одну какую-нибудь функцию и не может быть разбит на два модуля

- **Последовательная**

Модуль с такой связностью может быть разбит на последовательные части, выполняющие независимые функции, но реализующие совместно единственную функцию (например, оценка , а затем обработка данных)

- **Информационная**

Модуль с информационной связностью – это модуль, выполняющий несколько функций над одной и той же структурой данных, которая считается неизвестной вне этого модуля (применяется для реализации, например, абстрактных типов данных таких как стек, очередь и др.)

Связность модуля: Следует избегать

- **Временной связности**

- когда объединяются действия, связанные со временем (например, действия, которые должны быть выполнены в один и тот же момент времени)

- **Логической связности**

- когда в модуль объединяются действия по признаку их некоторого подобия (например, функции для проверки корректности входных данных для всей программы)

- **Случайной связности**

- когда действия объединяются произвольным образом

- **Процедурной связности**

- когда действия сгруппированы вместе только потому, что они выполняются в течение одной и той же части процесса

Сцепление модулей

Сцепление – мера относительной независимости модулей от других модулей. Независимые модули могут быть модифицированы без переделки других модулей. Чем слабее сцепление модуля, тем лучше.

Типы сцепления

Независимые модули – идеальный случай. В этом случае модули ничего не знают друг о друге. Взаимодействие модулей организуется через их интерфейсы, когда выходные данные одного модуля передаются на вход другого. Достичь такого сцепления очень сложно, и в большинстве случаев не нужно.

Типы сцепления

Сцепление по данным (параметрическое) – это сцепление, когда данные передаются модулю как значения его параметров или как результат его обращения к другому модулю для вычисления некоторой функции (Этот тип сцепления реализуется в языках программирования при обращении к функциям)

■ Разновидности этого сцепления:

- ✓ *Сцепление по простым элементам данных*
- ✓ *Сцепление по структуре данных (оба модуля при этом должны знать о внутренней структуре данных)*

Типы сцепления

Не рекомендуется использовать:

Сцепление по управлению – это сцепление, в котором один модуль управляет решениями внутри другого с помощью передачи флагов, переключателей и т.п. В этом случае один модуль должен достаточно хорошо знать структуру вызывающего модуля

Сцепление по внешним ссылкам – возникает, когда у одного модуля есть доступ к данным другого

Сцепление по кодам – возникает, когда коды инструкций модулей перемежаются друг с другом (внутренняя область одного модуля доступна другому)

Рутинность модуля

Рутинность модуля – это независимость модуля от предыдущих обращений к нему.

Будем называть модуль *рутинным*, если результат его работы зависит только от переданных параметров (а не от количества обращений)

Свойства модуля

- На модуль **можно ссылаться с помощью имени** модуля.
- Модуль должен иметь **один вход и один выход**
- Модуль должен быть **сравнительно невелик**
- Возможность **отдельной компиляции**
- Модуль **может вызвать другой модуль или сам себя**
- Модуль должен **возвращать управление** тому, кто его вызвал
- Модуль **не должен сохранять историю своих вызовов** для управления своим функционированием

Преимущества модульного программирования:

- Функциональные компоненты модульной программы **могут быть написаны и отлажены порознь**
- Модульную программу **проще проектировать, легче сопровождать и модифицировать**
- Становится **проще процедура загрузки** в оперативную память большой программы, требующей сегментации

Недостатки модульного программирования:

- *Может увеличиться время компиляции и загрузки.*
- *Может увеличиться время исполнения программы.*
- *Может возрасти объем требуемой памяти.*
- *Организация межмодульного взаимодействия может оказаться довольно сложной.*

Для современных компьютеров первые три недостатка несущественны.